

points: h) bookmarks: i) escape hatch (Amz logo)

CRUD via HTTP: Create-POST/Read-GET/Update-PUT/Delete-DELETE.HTTP is the way we communicate on the Internet. Data is transmitted by requests and responses the allow us to do those 4 operations.

HTTP request: 1. Request line(GET /cs571/2. Request headers (X-571-ID)3. Request Body. An HTTP request may have path(zaj) & query parameters(since):http://571.com/zaj/since=2020). For htmlfor/JSON body ("title":"zaj");): need additional HTTP header:Content-Type:"application/json"

Response body: 1. Status code2. Response headers3. Response Body(JSON when deal with data).

HTTP Status Codes: 100s informational, 200s successful, 300s redirection, 400s client error:400 bad request (sth on my end)/401 unauthorized (didn't have badge id)/404 not found/409 conflict (duplicates), 500s server error: 500 internal server error.

HTTPS:secure version w/ end-to-end encryption, used for: {"user":"zaj", "password":"aj123"}

Form.label has htmlfor, form.input has id. useRef is uncontrolled (we don't care). Const inputVal=useRef()>input ref={inputVal}> setVal(pre => pre+inputVal.current.value) to retrieve.

Pros & Cons: Generally recommends controlled compo be virtual DOM change cheap although many re-renders, uncontrolled means no control over the onChange property.

Credentialed Requests: When post register/login, it returns a response containing cookies allowing us to post/delete. Cookies contains a JS Web Token (JWT), temporary, all-access for authenticating w/ API. Be cs571.org different domain than localhost, must explicitly include credentials to every endpoint affecting authorization (login, logout, post)

Memoization:1. useCallback for function: const complicatedFunction = useCallback((arg) => {}, []) only create this function when components load (or change dependency), we still can use it with diff arguments 2.useMemo for value: const complicatedValue = useMemo(() => {}, []) only calculate this value when components load (or change dependency) 3. Memo: for purely functional components (doesn't have any states, so given same props => same output) memo(ticket): doesn't re-render if parent re-renders, only re-render when props change. We should find a balance otherwise messy to read code.

Accessibility: as many people as possible, primary focus individuals with disabilities. Disability: Impairment Anatomical perspective: sensory(loss vision)/physical(loss legs)/cognitive(Alzheimer). Common impairments: visual/motor/auditory/seizures/learning. Variability: severity(self/caretaker) and structure (colorblind/peripheral-only vision/no light perception)/Temporal perspective: Permanent(one arm)/Temporary(broken arm)/Situational(busy).Medical model of disability: personal attribute Social model: barriers in society. Mismatch btw abilities and environment: left-handed people.

Universal design: used w/o specialized design/adaptation. Principles: 1. Equitable use (identical/equivalent to all users,no segregate, provision for privacyce, appeal to all) 2. Flexible in use (accommodate wide range of preferences and abilities) 3. Simple and intuitive use (easy understand w/o knowledge, be intuitive) 4. Perceptible information (contrast btw essential & surroundings, different modes for essential, eg. Subtitle transcript) 5. Tolerance for error (warning & minimize accidents, eg. Wrong password/) 6. Low physical effort (eg. short cut/7. Size and Space for approach and use (regardless of users' body size etc, eg phone size)

Assistive technology: screen readers/ screen magnification (enlarge/text readers/braille for web (keyboard for blind). Alternative input: head/mouth pointers/ eye tracking/ single-switch/speech input/ alternative & augmentative communication (options)

Customized React hooks: minimized repeat Deploy: npm run build generates build bundle (deliver HTML JS CSS to client)

NPM & Node.js: package.json: details about the project. Package-lock.json: details about the dependencies. Npm install: installs dependencies in node_modules. Npm run dev: run w/ hot-reloading. Node is the runtime engine, NPM is the package manager.

True native: specifically for the device (Swift for iOS, Kotlin for Android). Pros: Optimized Apps, fine-grained control. Cons: Expensive, little code reuse btw Android and iOS, less sense of Abstraction (1 swift team & 1 Kotlin team). Alternative: responsive design webpage/webview with apache cordova (webpage inside an app)/Cross-platform React Native (a JS framework). Android/iOS shared similarities: ViewGroup/UIWebView/ImageView/UIImageWebView, TextView/UITextView.

```
const BookStack = createNativeStackNavigator();

<NavigationContainer>
  <BookStack.Navigator>
    <BookStack.Screen name="Bookstore" component={BookStoreScreen}/>
    <BookStack.Screen name="Book" component={BookScreen}/>
  </BookStack.Navigator>
</NavigationContainer>
```

useCallback:fuct useMemo:calculated values memo: components
create bundle -> deploy (apache) Node is the runtime engine NPM is the package manager
True Native: specifically for iOS/Android; Organic experience, optimized apps; Con:真. little code reuse, 更不抽象. Alt: no mobile, WebView, or 跨平台:React Native **React Native:** JS framework(ReactJS is library), no:DOM,CSS, Bootstrap, sessionStorage, localStorage, cookies. yes: declare prog, use JSX with RN, React hooks, styling(JS stylesheets&inline.no CSSstyles) **Image**(x img; source, x src) **Buttons**<Button title=“hi”(a prop) onPress{doA}(x onClick)/> **By Platform:** components that utilize platform capabilities & not available in other platforms. **Dimensions:** obtain screen size. **Switch:** value: on/off, onValueChange. **ScrollView:** must be in a view that is flex-ible. **Card:** no card, use 3rd-party r-n-paper/diy. **Animated:** animation for: View,Text,Image,ScrollView,FlatList, SectionList. e.g. <Animated.View></A.V> **Animated.timing,** .spring, .decay. Animated.Value used with useRef. Control many animations using: A.parallel, A.sequence, A.loop. start&stop() apply to the set of animations. A.value changed by i.e. A.add(v1,v2). **Navigation:** @react-navigation/bottom-tabs or drawer or native-stack. Must be nested in NavigationContainer. created via func createBottomTabNavigator() etc. Navigator can be styles & nested. **useNavigation:** custom R hook for navigate. **Stack Nav:** can push screen onto history stack via navigation.push(screenName name of the screen to navigate to, param optional object of parameters to pass to the receiving screen) param received as props.route.params. Navigators can be nested stacks in Tabs/Drawer/Tabs in Drawer. etc. **Switch navigation:** isSignedIn? (<Stack.Screen name=“home” component = {homeScreen}/>);(<Stack.Screen name=“SignIn” component = {signInScreen}/>)**Modal:** secondary window; manage open/closed via state variable, e.g. const [modalVisible, setModalVisible] = useState(false). **properties:** animationType: ‘slide’, ‘fade’,‘none’; onShow & onRequestClose: callback func; transparent: true/false; visible: true/false handled by state variable. **accessible** attribute indicates if a component is an accessibility element, if so groups its children in a single selectable element. **accessibilityLabel:** screenreader description. **accessibilityHint:** help user understand what will happen. **Secure Storage:** JSON Web Token (JWT): consist of a header, claim, signature. **Authentication:** web: interface with the user’s browser, prefer HTTP-only cookies. mobile: interface w. user’s operating system, prefer os-level secure encrypted storage. when fetching: web: use include: “credentials”, use header: {“Authorization”: “Bearer <JWT>”}. **expo-secure-storage JWT:** getItemAsync(key), setItemAsync(key, val), deleteItemAsync(key). can store a JWT on Android and iOS up to 2kb per value to persistent device storage **Gestures:** r-n’s Button onPress. r-n-paper’s Card onPress & onLongPress. r-n-elements’ Slider onSliding. r-n-maps’ Marker onDrag. **Sensors:** Use expo-sensors instead of react-native-sensors. Accelerometer, Barometer气压计, Gyroscope陀螺仪, LightSensor, Magnetometer, Pedometer计步器. not all device have sensors **Deployment** use expo application services: npm install -g eas-cli; eas build -p android/ios .apk is to google play store, .ipa to apple store

dialogflow: agents: high-lvl containers. **Intents**(goal of user):training phrases(things to express an intent)/parameters(eg, time/location)/responses(system, can have text/button etc). fallback(“couldn’t get that”). Parameter: variable by user (city name). Entity type: type of variable (geolocation). **Entity:** specific value(Madison). Entities can: have synonyms/fuzzy- matched/automatically expanded(Chicago probably is another entity). Dialogflow Advanced: 1. **Webhook** fulfillment: Dialogflow makes a POST request to our server to. Steps: place token.secret, npm start, npm tunnel, ngrok: reverse-proxy service **Async:** we promise sth in the future, if we fulfill, then we do sth; otherwise we catch and handle error. Async/await: an equivalent way. **Await** must inside an async function/top-lvl of the program, waiting for right- hand-side to complete. Every async function returns a Promise. A synchronous function may spawn async behavior, an async function always happens asynchronously 2. **Context:** can be used in follow-up intents, allows us to control the order of intents and define different behaviors for intents w/ same training phrases. It has lifespan defined by time/turns. A content can be added or removed by an intent. 3. **Events:** trigged by actions users take on platforms(eg Google assistant) that Dialogflow interacts with. 4. **Integrations:** Dialogflow can be integrated as an iframe.

fullstack software stack: JS and React for frontend, JS and Express for backend. **Setup:** npm create vite@latest my- app -- --template react, npm install bootstrap react- bootstrap (Bootstrap also has additional setup). **Backend** persists data by SQLite creates a .db file on my machine, not a hosted database run by online server. **Deploy:** a server is just another computer w/ a bunch of holes in its firewall (allow others to connect). Isolate: security & parity (works on everyone’ machine) reasons, use a virtual machine or a container. **Docker:** dockerfile + my code to(recipe for create my code in isolated environment) build docker image, docker image (compiled recipe for environment and my code) can run in dock container (my machine, your machine, cloud). Other considerations: use CI/CD platform to create a build-and-deploy pipeline (press buttons)/use HTTPS for security/Completely optional to buy a domain name.

Prototyping Paper prototyping: Mock up design ideas by sketching pages, simulate how the system would respond to user input. **Wireframes:** Lo-fi prototypes of the makeup of a design in terms of its structural components. Hand-drawn/digital **Annotations:** Labels, explanations, notes providing further info on goals, content, functioning of the design elements illustrated on wireframes. **Interactive prototyping:** realistic prototypes of navigation and/or structural components of the design idea by creating a series of screens with design elements. Link the screens and simulate transitions. **Native prototyping:** Implement & test design ideas on target technology platform of design.

Prototyping Theory 3D model of prototyping: represent 3d of a design idea: role, look & feel, implementation, can be various levels of fidelity. Their integration makes a working prototype / pre-alpha product. **Role:** Represent the function that the system serves in the user’s life. **look & feel:** simulates the sensory experience of user while using the system. **implementation:** includes the technical capabilities that enable the system to perform its function. **Integration:** Represents the complete user experience with the system as envisioned in the conceptual design. **Prototyping Scope:** horizontal: broad view of entire sys. vertical: focusing on single feature. **Prototyping Strategies** **Throwaway prototyping:** Rapid prototyping to explore design ideas, most lo-fi & paper **Evolutionary(or breadboard) prototyping:** incrementally builds prototypes, test & refine until reach maturity. **Incremental prototyping:** divide system based on design specifications, incrementally building each slice then integrated into the overall system. **Extreme prototyping:** 1: building a static prototype, 2: build fully functional interactive components, simulating services 3: implement services. **Fidelity:** level of detail in prototype. higher fidelity = narrower feedback. **lo-fi pro:** lower cost; prevent premature wedding to specific ideas; enables exploring, communicating, testing of conceptual designs; helps designers identify structural, navigation, and organizational issues; allows rapid eval. **lo-fi con:** requires a facilitator; limited when identifying breakdowns; lacks sufficiently low-level specs for development; limited sense of feasibility.

MobileDesign **Mobile input** **Direct Manipulation Input:** in web & desktop, dmi use mouse input that is mapped to screen using relative mapping, on mobile, its absolute, user directly interacts with screen elements. **Multi-touch Gestures:** idiomatic gestures for specific functions. **Tap:** maps to click 2 **tap:** zoom in/out, select item in accessible mode, select text **long-press:** open contextual menus, preview content, enable edit mode. **drag/swipe:** scroll through content, move object, adjust controls, most commonly used. **triple-swipe:** mapped to specific OS-level activity, e.g. undo, redo. **2 finger pinch & spread:** shrink/expand visual elements. **3 finger pinch & spread:** OS-level actions, e.g. copy, cut, paste. **motion gesture:** gesture by moving mobile device, e.g. shaking. application specific/ customizable at os level **Microinteractions:** contained product moments for a single use case e.g. like button / pull to refresh. **4 parts: trigger** events that initiate the MI, manual(intentional) or automatic(when conditions are met) **rules** determine what happens when MI trigger. **feedback** info user see/hear/feel **loop & mode** meta rules that change mi rules and feedback, e.g. snooze a reminder. loop determine mi length & if repeats. **Mobile design patterns:** stacks vertically organize elements e.g. search result. **Screen carousels:** 往右滑5h后的天气. **drawers:** links for app’s nav. **list n card:** 通讯录 carousel: 电影 swimlane: 网飞 card: appstore frontpage bars: strips w. buttons. search, sort, filter: landing page, guided tour: advanced direct manipulation: apps e.g. image editors. Panes and panels: apple pencil top panels. AR/VR.

VUI Experience Prototyping Conversational interfaces(CI) mostly less effective, efficient, satisfactory than graphical ui. **CI aspects:** speech recognition + spoken language understanding + dialog management + response generation + text-to-speech synthesis **Pro:** streamline actions in a conver. paradigm; when driving; more accessible w disability. **Griecan Maxims:** quality, quality, relevance, manner. **multimodality:** most CI is, utilize multiple modality **multimodality principle:** take advantage other modality. **interaction paradigm:** CI follow different paradigms depending on the context. **command-and-control interfaces:** speech input mapped to specific system. **Conversational**

Interfaces: interaction with the system has human convo characteristics. **Turn-taking.** **Conversational Markers:** indicate the state of convo. timelines, acknowledgements, positive feedback. **confirmations:** let user know CI understood. Explicit confirmation: requiring the user to confirm. Implicit confirmation: Letting user know what was understood **error handling:** 1. no speech detected; detected, not recognized; recognized, but does wrong thing; incorrectly recognized. **Experience Prototyping:** prototyping the holistic experience of interacting with a product. understanding existing user experiences and context; explore and evaluate design idea; communicate ideas to an audience. **what:** system behavior, user behavior, interactions w. context. **how:** 1. define context. 2. develop scenarios. 3. identify design goals. 4. set up environment. 5. bodystorming. 6. develop insight.

Usability Evaluation(UE) **UX design steps:** empathize, define, ideate, prototype, test, implement. **testing-based UE:** empirical, i.e. based on data. **five-e of usability:** effective, efficient, engaging, error tolerant, east to learn. **formative testing:** testing done throughout the design process to diagnose design problems, small num of users, repeatedly, informs design improvements. **summative testing:** done at the end of design process to establish the baseline usability. large num of users, comparative testing, large num of metrics. **lab testing,** **field testing:** **guerrilla testing:** set up in public with passerby, low cost. **remove testing:** for hi-fi prototype, moderated or unmoderated. **dimensions of usability testing:** why(goals), what(scope, task/scenarios), how(approach, metrics), who(user subgroups, study team) **qualitative data:** observations of user actions and behavior, comments, and answers to questions; **quantitative data:** measurements of user performance, error, and ratings of the design. **measures:** 1. performance metrics(task success, time on task, errors, efficiency, learnability) 2.self-report metrics(SUS/USE scale) 3. issue-based metrics: problems user encounters.

Heuristic Evaluation **expert-review-based methods:** follow established protocols. **HE:** having a small set of evaluators examine the interface. 1.find 3-5 experts. 2. review individually. 3.merge findings **protocols:**1.visibility of system status, i.e. keep user informed. 2.match between system and the real world 3.user control and freedom 4.consistency & standards 5.error prevention. 6.recognition rather than recall. 7.flexibility & efficiency of use. 8.aesthetic & minimalist design. 9.help users recognize, diagnose, recover from errors. 10.help & documentation. **pro:** inexpensive & intuitive; can be used any time; effective at early stage of design; training tool for designers. **con:** doesn’t capture all aspects of usability; not a comprehensive understanding of the interaction; might discourage user testing; may result in false positive. Seventeen heuristics that fall into **five broad categories:**1. general 2. conversational style 3. guiding, teaching, offering help 4. feedback & prompts 5. errors **cognitive walkthrough** needs: a prototype, a user profile, set of tasks, sequences of actions. **pro:**powerful for: walk-up-and-use interfaces, new concepts/forms of interaction, systems designed for various user profiles; can be performed any time/stage. **con:** focus only on discoverability/learnability, best when used with usability testing

```
Animated.Value is used in combination with useRef.
const opVal = useRef(new Animated.Value(0)).current
useEffect(() => {
  Animated.timing(opVal, {
    toValue: 1,
    duration: 5000,
    useNativeDriver: true,
  }).start();
}, []);
return (
  <View>
    <Animated.View
      style={{
        height: 100, width: 100, opacity: opVal
        backgroundColor: "cyan",
      }}
    />
  />
)

```

```
<NavigationContainer>
  <ToolDrawer.Navigator>
    <ToolDrawer.Screen name="Landing" component={LandingScreen}/>
    {
      tools.map(tool => {
        return <ToolDrawer.Screen key={tool.name} name={tool.name}>
          {(props) => <ToolScreen {...props} {...tool}/>}
        </ToolDrawer.Screen>
      })
    }
  </ToolDrawer.Navigator>
</NavigationContainer>

```

```
async function doGetDateTIme(req, res) {
  const chatroomName = req.body.queryResult.parameters.chatroom
  const resp = await fetch("https://cs571.org/api/f23/hw11/messages?chatroom=" + chatroomName + "&page=1", {
    headers: { "X-CS571-ID": CS571.getBadgerId() }
  })
  const resBody = await resp.json()
  const firstPostDate = new Date(resBody.messages[0].created)
  const formattedDate = firstPostDate.toLocalDateToString();
  const formattedTime = firstPostDate.toLocalTimeToString();
  res.status(200).send({
    fulfillmentMessages: [
      {
        text: {
          text: [
            "The last message in $(chatroomName) was posted on $(formattedDate) at $(formattedTime)!"
          ]
        }
      }
    ]
  })
}

```

const SocialTabs = createBottomTabNavigator();

```
<NavigationContainer>
  <SocialTabs.Navigator>
    <SocialTabs.Screen name="NewsFeed" component={NewsFeedScreen}/>
    <SocialTabs.Screen name="Notifications" component={NotificationScreen}/>
    <SocialTabs.Screen name="AboutMe" component={AboutMeScreen} />
  </SocialTabs.Navigator>
</NavigationContainer>

```

Heuristics for Conversational Interfaces: 17 heuristics into 5 broad categories: 1. General 2. Conversational style 3. Guiding, teaching and offering help 4. Feedback and prompts 5. Errors. Eg. #5 Pay attention to what the user said and respect the user’s context. #12 Confirm input intelligently(implicitly thru results, confirm irreversible/critical action explicitly and allow undo after confirmation) 2.Cognitive Walkthru: a specialist assess learnability and discoverability by posing and answering questions. We need: A prototype/A user profile/Set of tasks/Sequences of actions. Questions: 1. Will the user try and achieve the right outcome(Does design support user’s mental modal/knowledge to achieve outcome? Otherwise they may not even use the app)? 2. Will the user notice that correct action is available to them(Give hints to users some function is available, some not)? 3.Will the user associate the correct action w/ the outcome they expect to achieve? 4. If correct action is performed, will the user see that the progress is being made towards their intended outcome? Pros: Powerful for walk-up-and-use interfaces/New concepts of interaction/Systems designed for various user profiles/can be performed frequently and any stage of the design process. Cons: Focuses only on only on discoverability/learnability/Very narrow not powerful, best when used with usability testing together.

```
animationType:'slide'
transparent:true
visible={modalVisible}
<view style={styles.centeredView}>
  <view style={styles.modalView}>
    <text style={fontStyle: 24, paddingBottom: 10 }>Create a Post/Text</text>
    <text style={fontStyle: 16 }>Title/Text</text>
    <textInput
      style={styles.input}
      placeholder="Enter Title"
      onChangeText={(text) => setTitle(text)}
    />
    <text style={fontStyle: 16, paddingTop: 10 }>Body/Text</text>
    <textInput
      style={styles.input}
      placeholder="Enter Body"
      onChangeText={(text) => setContent(text)}
    />
    <view style={styles.pagination}>
      <button disabled={title || content} title="Create Post" onPress={() => {
        handlePost();
        setModalVisible(modalVisible)
      }} />
      <button color="crimson" title="Cancel" onPress={() => setModalVisible(modalVisible)} />
    </view>
  </view>
</view>
</Modal>

```